

CENTRO UNIVERSITÁRIO INTERNACIONAL UNINTER
ESCOLA SUPERIOR POLITÉCNICA
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
DESENVOLVIMENTO WEB BACK END

ATIVIDADE PRÁTICA
API REST – BAOZI STORE

VINÍCIUS BARBOSA SILVA – RU: 5474751

SÃO PAULO – SP
2026

ATIVIDADE PRÁTICA

1. **Descrição de uma situação fictícia**
2. **Diagrama caso de uso**
3. **Especificação da API desenvolvida**
4. **Prints do Postman**
5. **Link do repositório (GitHub ou similar) contendo o projeto(código fonte)**

1. Descrição de uma situação fictícia

A **Baozi Store** é uma pequena loja que vende **pão chinês**. Para melhorar a organização do negócio, foi criado um sistema simples para controlar clientes, produtos e pedidos. Um cliente chamado **Vinícius5474751** realizou seu cadastro no sistema. O produto vendido pela loja chama-se **Baozi Tradicional** e é vendido por unidade. Em um determinado momento, o cliente realizou um pedido de **6** unidades do produto. O sistema registra o cliente, o produto comprado e a quantidade solicitada, facilitando o controle da loja.

Para facilitar a conferência com os prints do Postman (seção 4), os dados da situação descrita são resumidos abaixo. Esses mesmos valores são enviados nas requisições e aparecem nas respostas da API.

Entidade	Campo	Valor na situação fictícia
Cliente	nome	Vinícius5474751
Cliente	clienteDesde	2026-10-06
Produto	nome	Baozi Tradicional
Produto	preco	R$ 8.50 por unidade
Produto	estoque	true (disponível)
Pedido	cliente / produto	Vinícius5474751 / Baozi Tradicional
Pedido	quantidade	6 unidades

Valor total do pedido (informativo): 6 × R$ 8.50 = **R$ 51.00**.

2. Diagrama de caso de uso (UML)

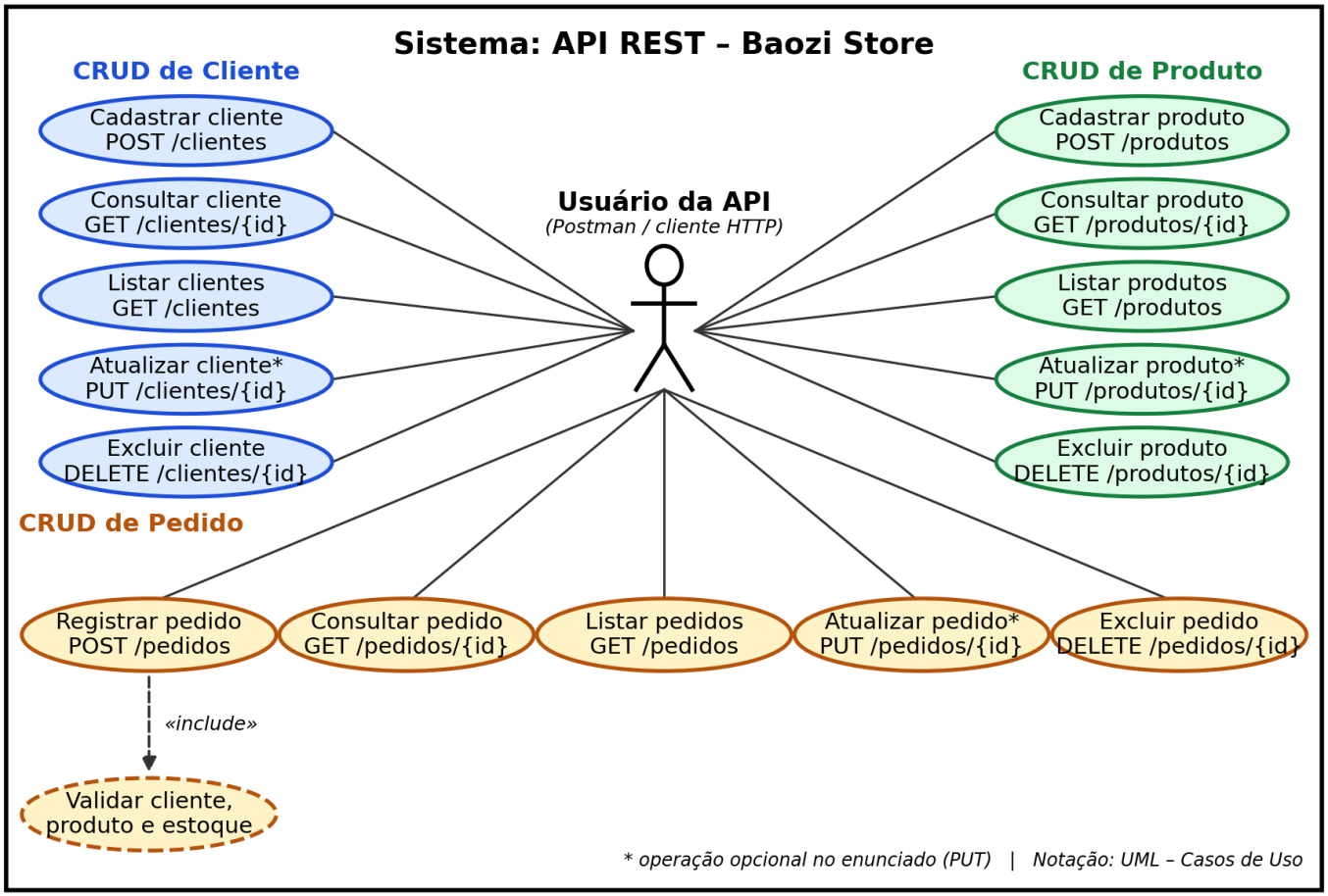


Figura 1 – Diagrama de Casos de Uso da API REST Baozi Store

Notação UML padrão: o **ator principal** é o **Usuário da API** (fora da fronteira do sistema) e os **casos de uso** (elipses) ficam dentro da fronteira *API REST – Baozi Store*, cobrindo o CRUD de Cliente, Produto e Pedido. A atualização (PUT) é opcional no enunciado e foi implementada como extra. O relacionamento *«include»* indica que registrar um pedido sempre executa a validação do cliente, do produto e do estoque.

3. Especificação da API desenvolvida

3.1 Tecnologias e arquitetura

A API foi desenvolvida em **Java 17** com **Spring Boot**, usando **Spring Data JPA** (Hibernate) para a persistência em **banco relacional** e **JSON** como formato de troca de dados, seguindo o padrão **MVC do Spring**. O projeto está organizado nos pacotes abaixo (arquitetura mínima exigida + *dto*):

Pacote	Responsabilidade
br.com.baozistore.model	Entidades JPA: Cliente, Produto e Pedido
br.com.baozistore.repository	Interfaces JpaRepository (acesso ao banco)
br.com.baozistore.controller	Controllers REST (ClienteController, ProdutoController, PedidoController) e tratamento de erros
br.com.baozistore.dto	PedidoRequest: corpo de entrada do pedido (clienteId, produtoId, quantidade)

3.2 Banco de dados

O banco padrão é o **H2** (relacional, em arquivo `./data/baozidb`), que dispensa instalação. Também há o perfil **mysql** (`application-mysql.properties`) para MySQL/MariaDB, ativado com `-Dspring-boot.run.profiles=mysql`. As tabelas são criadas automaticamente pelo Hibernate (`spring.jpa.hibernate.ddl-auto=update`). A tabela *pedido* possui duas chaves estrangeiras: `cliente_id` → `cliente.id` e `produto_id` → `produto.id` (relacionamentos N:1, conforme o DER do enunciado).

3.3 Entidades criadas e campos

Cliente (tabela `cliente`)

Campo	Tipo Java	Coluna	Regras
id	Long	id	Chave primária, auto incremento
nome	String	nome	Obrigatório (@NotBlank)
clienteDesde	LocalDate	cliente_desde	Se omitido, assume a data atual

Produto (tabela `produto`)

Campo	Tipo Java	Coluna	Regras
id	Long	id	Chave primária, auto incremento
nome	String	nome	Obrigatório (@NotBlank)
preco	BigDecimal	preco	Obrigatório, maior que zero, DECIMAL(10,2)
estoque	Boolean	estoque	Obrigatório; true = disponível, false = sem estoque

Pedido (tabela `pedido`)

Campo	Tipo Java	Coluna	Regras
id	Long	id	Chave primária, auto incremento
clienteId	Long	cliente_id (FK)	Obrigatório; referencia cliente (@ManyToOne)
produtoId	Long	produto_id (FK)	Obrigatório; referencia produto (@ManyToOne)
quantidade	Integer	quantidade	Obrigatório, mínimo 1

Na resposta JSON, o pedido traz `clienteId` e `produtoId` e também os objetos `cliente` e `produto` completos, o que permite ver nome, produto e quantidade no mesmo print.

3.4 Endpoints implementados

Todos os endpoints trocam dados em JSON (`Content-Type: application/json`). URL base: `http://localhost:8080`.

Clientes

Método	Endpoint	Descrição	Sucesso / Erros
POST	/clientes	Cria cliente. Corpo: nome, clienteDesde	201 Created / 400
GET	/clientes/{id}	Consulta por ID	200 OK / 404
GET	/clientes	Lista todos	200 OK
DELETE	/clientes/{id}	Apaga por ID	204 No Content / 404 / 409
PUT (opc.)	/clientes/{id}	Atualiza (opcional)	200 OK / 400 / 404

Produtos

Método	Endpoint	Descrição	Sucesso / Erros
POST	/produtos	Cria produto. Corpo: nome, preco, estoque	201 Created / 400
GET	/produtos/{id}	Consulta por ID	200 OK / 404
GET	/produtos	Lista todos	200 OK
DELETE	/produtos/{id}	Apaga por ID	204 No Content / 404 / 409
PUT (opc.)	/produtos/{id}	Atualiza (opcional)	200 OK / 400 / 404

Pedidos

Método	Endpoint	Descrição	Sucesso / Erros
POST	/pedidos	Cria pedido. Corpo: clienteId, produtoId, quantidade	201 Created / 400
GET	/pedidos/{id}	Consulta por ID	200 OK / 404
GET	/pedidos	Lista todos	200 OK
DELETE	/pedidos/{id}	Apaga por ID	204 No Content / 404
PUT (opc.)	/pedidos/{id}	Atualiza (opcional)	200 OK / 400 / 404

Regras de negócio e erros: o pedido só é criado se o cliente e o produto existirem (senão **404**) e se o produto estiver em estoque (senão **422**). Um cliente ou produto com pedidos vinculados não pode ser apagado (**409 Conflict**), preservando a integridade referencial. Dados inválidos retornam **400** com a lista de campos com erro.

3.5 Exemplos de corpo das requisições (JSON)

Requisição	Corpo (JSON)
POST /clientes	`{"nome": "Vinícius5474751", "clienteDesde": "2026-10-06"}`
POST /produtos	`{"nome": "Baozi Tradicional", "preco": 8.50, "estoque": true}`
POST /pedidos	`{"clienteId": 1, "produtoId": 1, "quantidade": 6}`

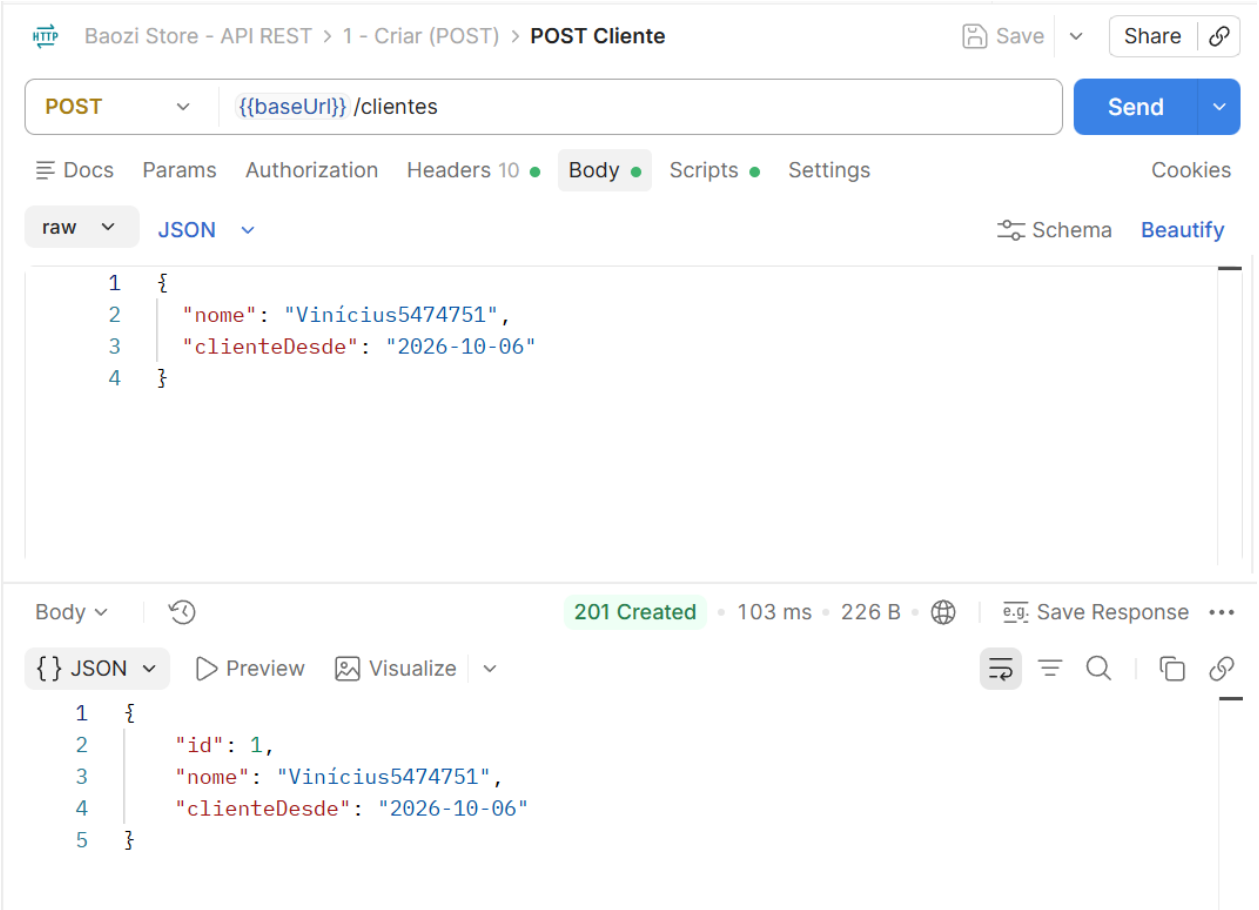
4. Prints do Postman

Os testes foram executados com a API em `http://localhost:8080`. Em todos os prints aparecem as informações da situação fictícia: cliente **Vinícius5474751**, produto **Baozi Tradicional** e pedido de **6** unidades.

4.1 POST de cada entidade

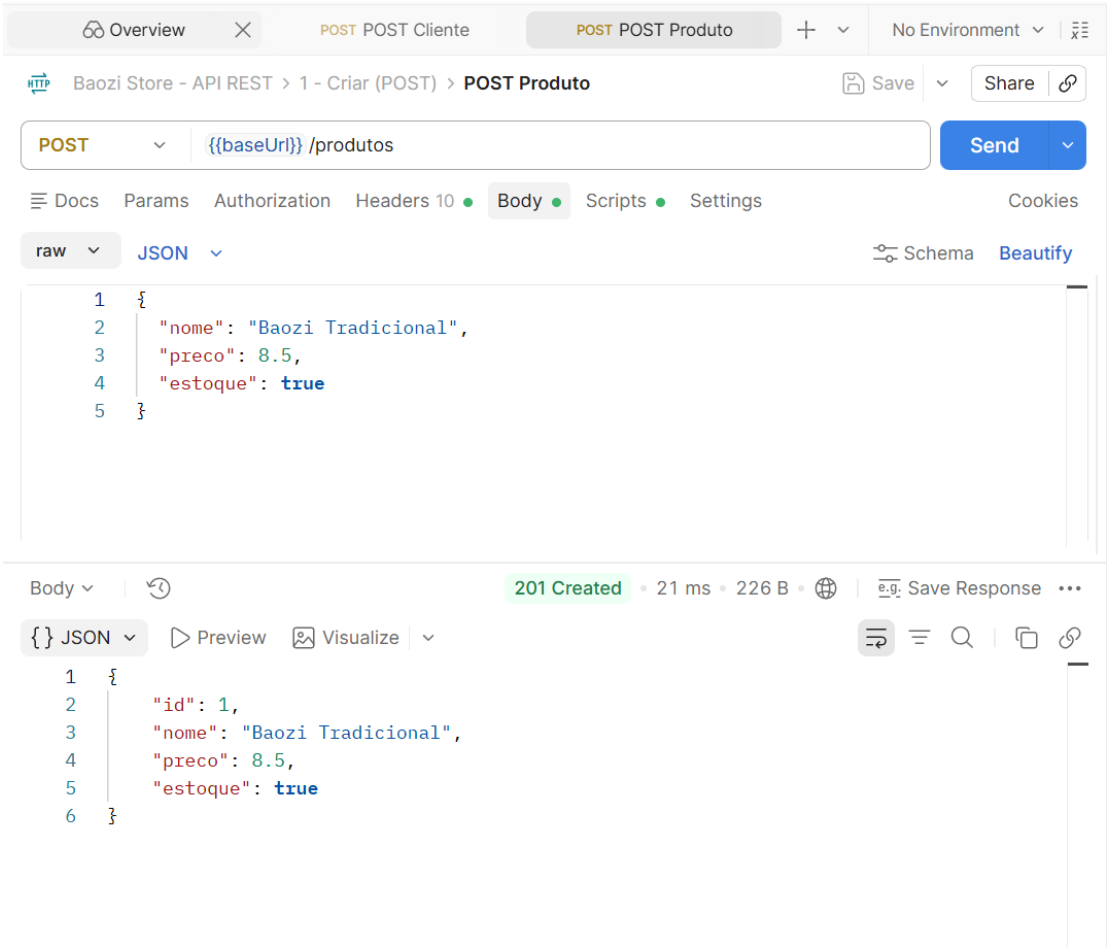
4.1.1 POST – Criar Cliente

Requisição: `POST /clientes`
Resultado esperado: status 201 Created; JSON com id e nome “Vinícius5474751”.



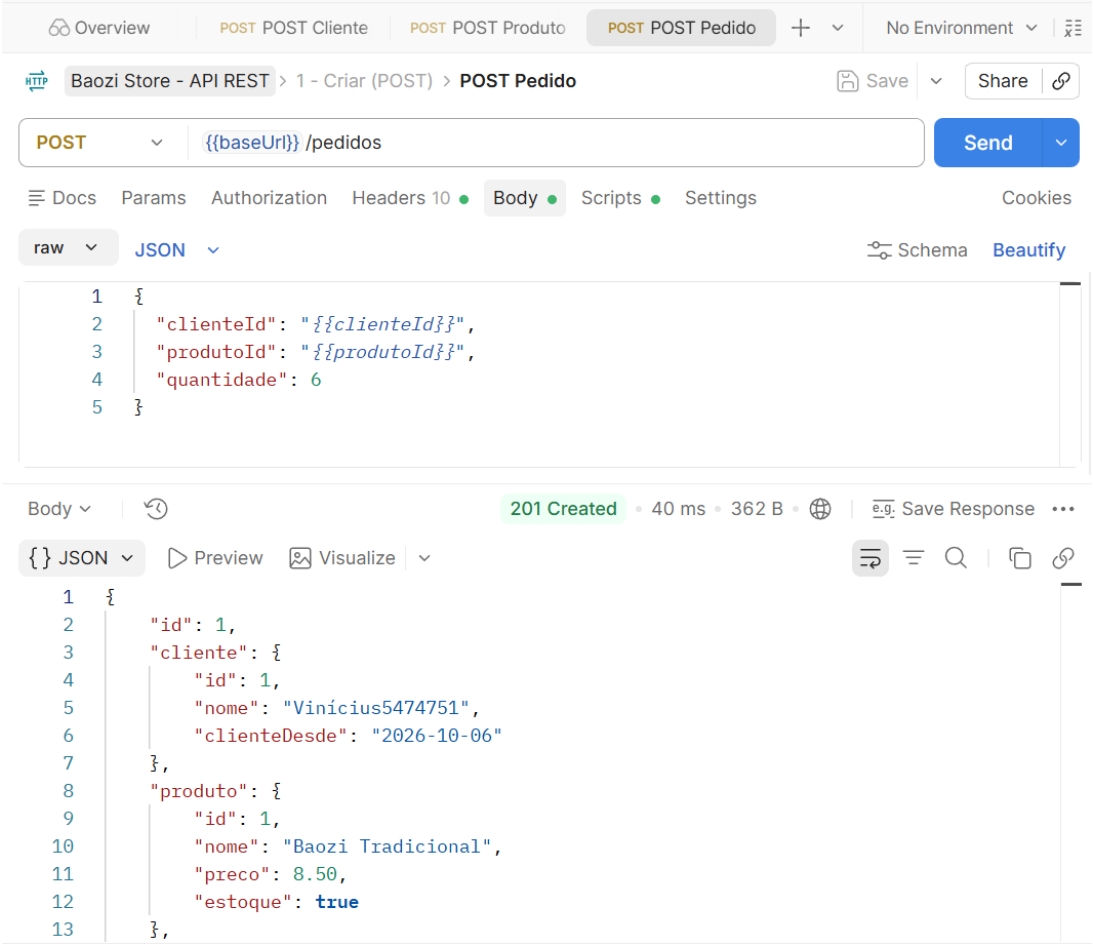
4.1.2 POST – Criar Produto

Requisição: `POST /produtos`
Resultado esperado: status 201 Created; JSON com id, nome “Baozi Tradicional”, preço 8.50 e estoque true.



4.1.3 POST – Criar Pedido

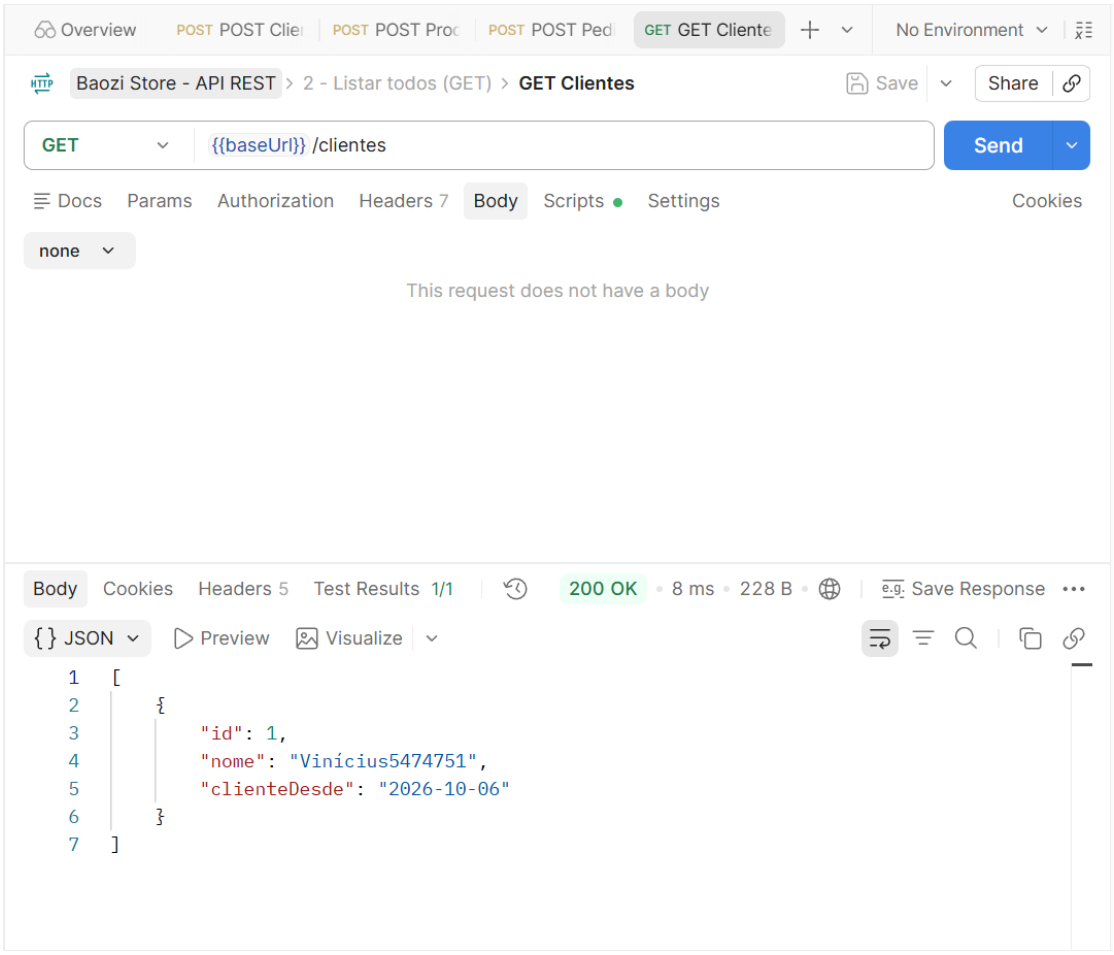
Requisição: `POST /pedidos`
Resultado esperado: status 201 Created; JSON com cliente “Vinícius5474751”, produto “Baozi Tradicional” e quantidade 6.



4.2 GET geral (listagem de todas as entidades)

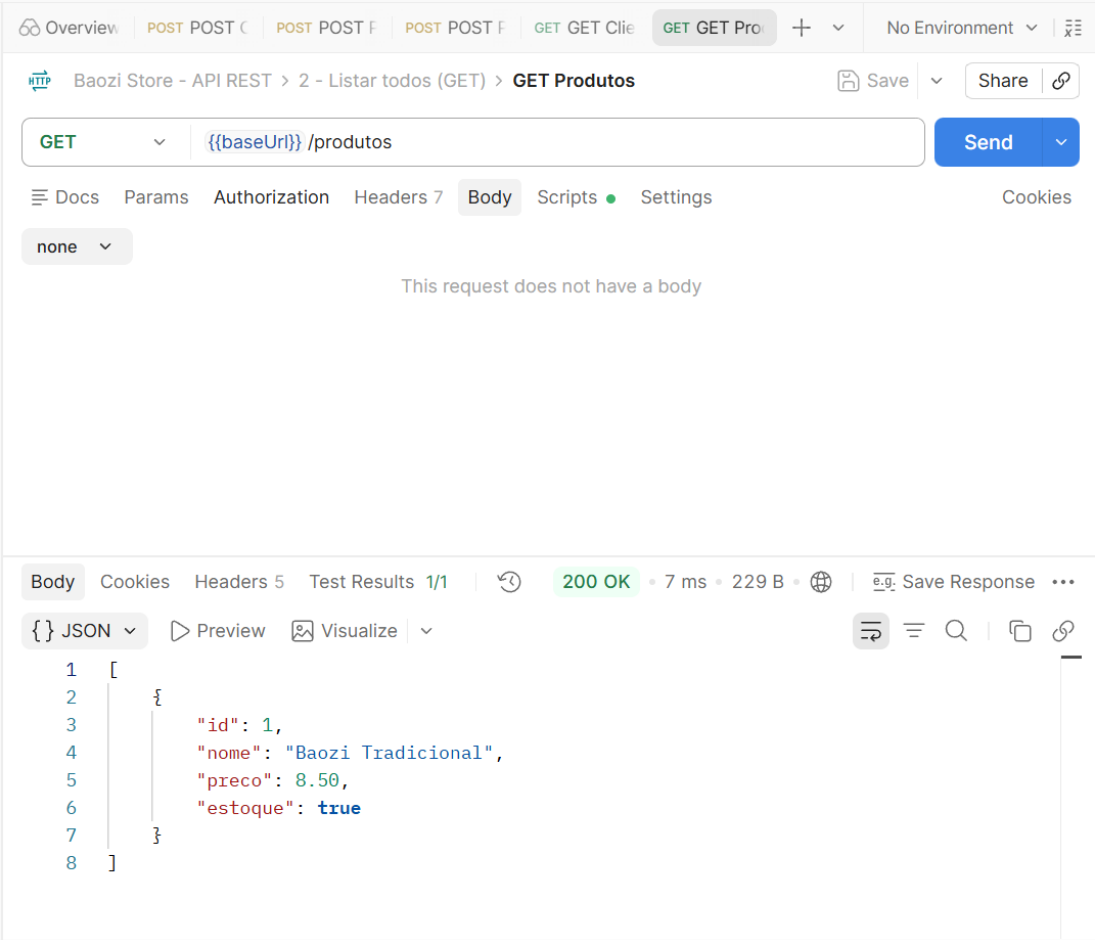
4.2.1 GET – Listar Clientes

Requisição: `GET /clientes`
Resultado esperado: status 200 OK; lista com o cliente cadastrado.



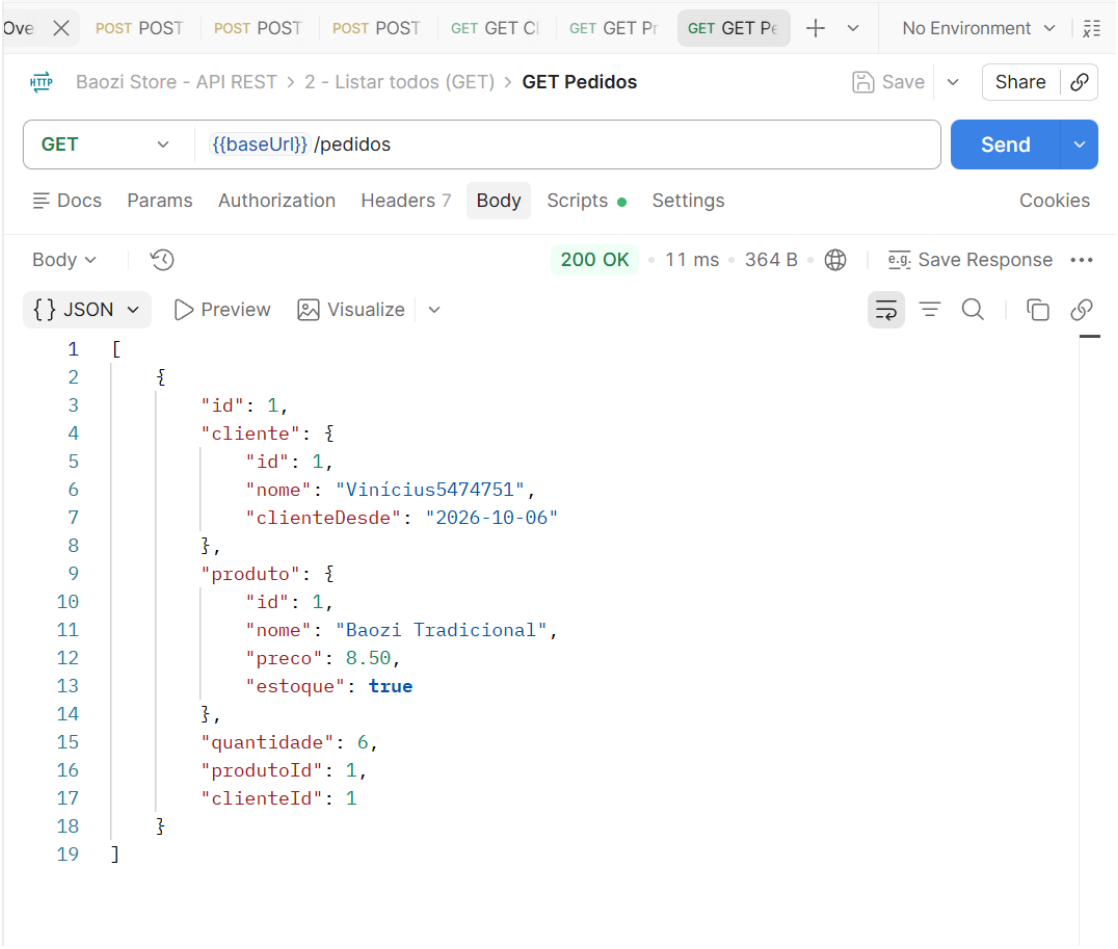
4.2.2 GET – Listar Produtos

Requisição: `GET /produtos`
Resultado esperado: status 200 OK; lista com o produto cadastrado.



4.2.3 GET – Listar Pedidos

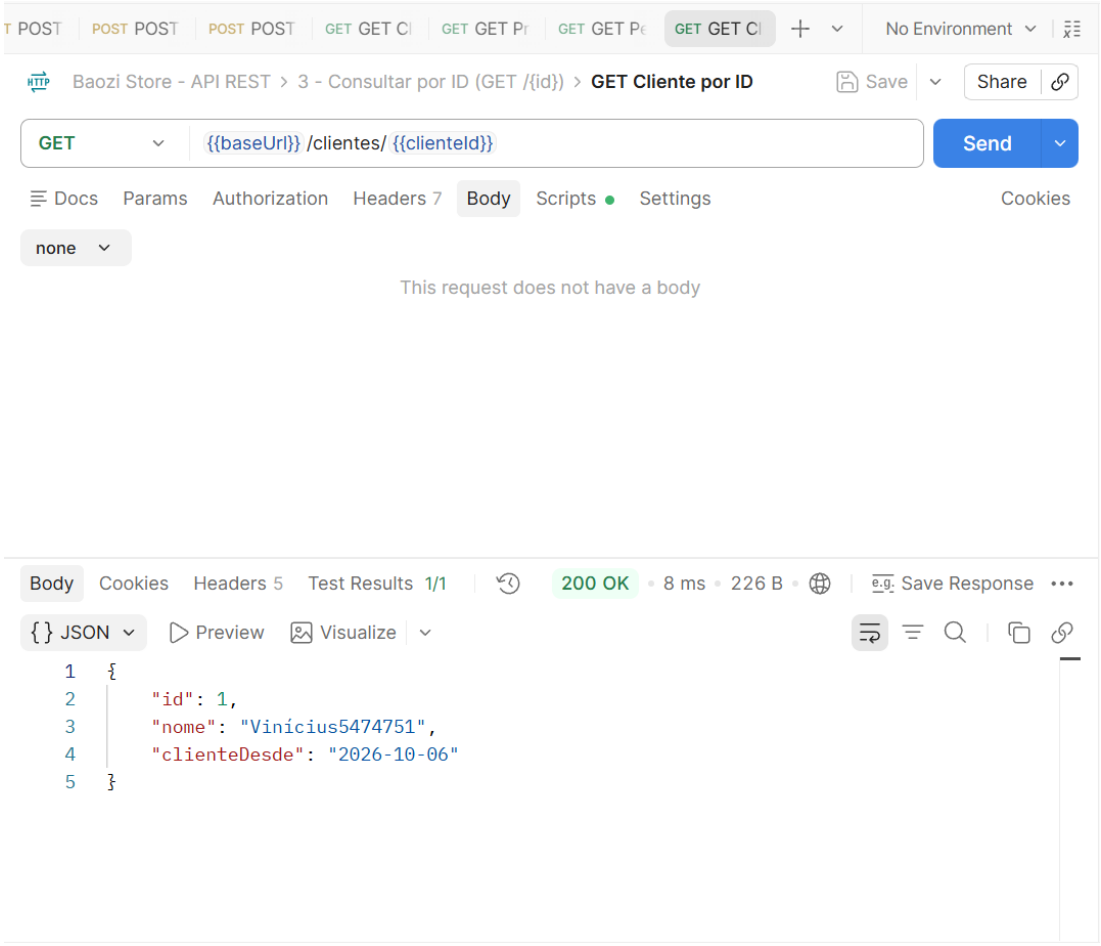
Requisição: `GET /pedidos`
Resultado esperado: status 200 OK; lista com o pedido registrado.



4.3 GET por ID

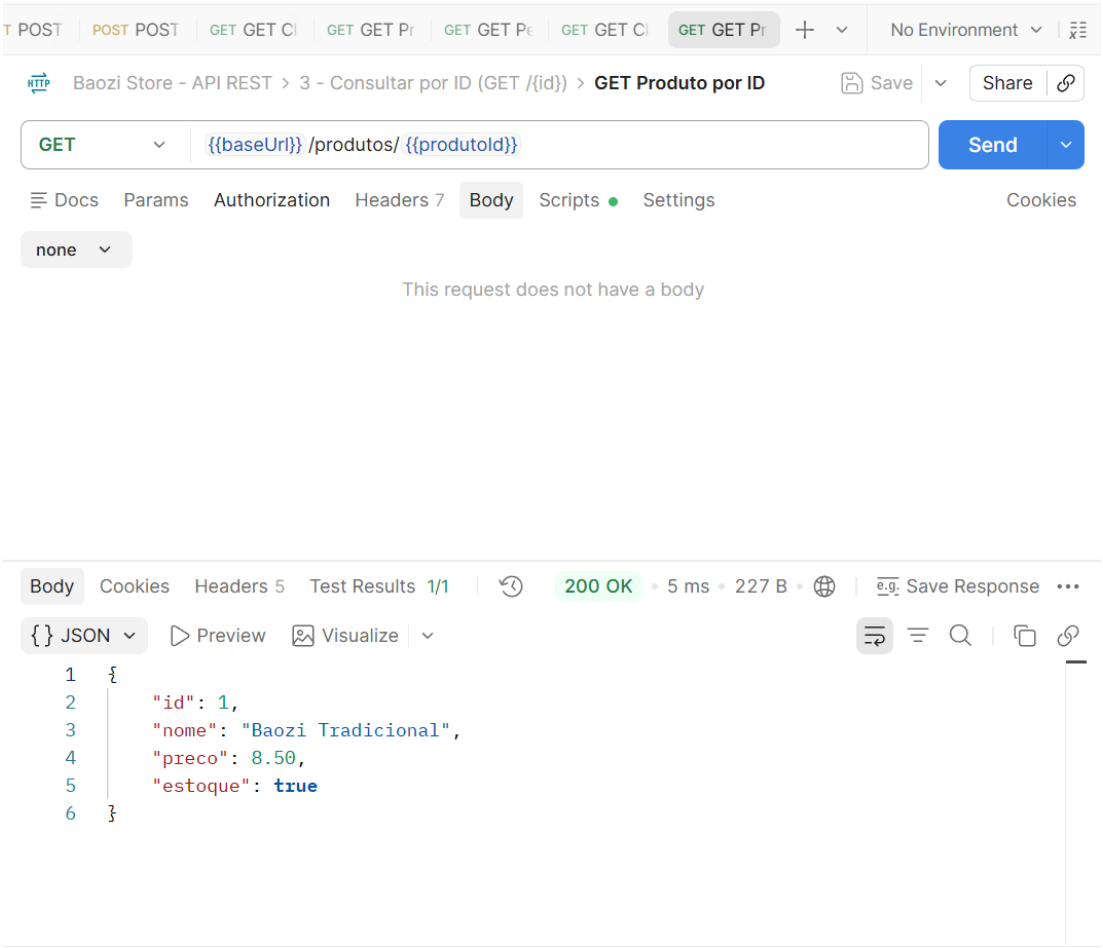
4.3.1 GET – Consultar Cliente por ID

Requisição: `GET /clientes/1`
Resultado esperado: status 200 OK; cliente “Vinícius5474751”.



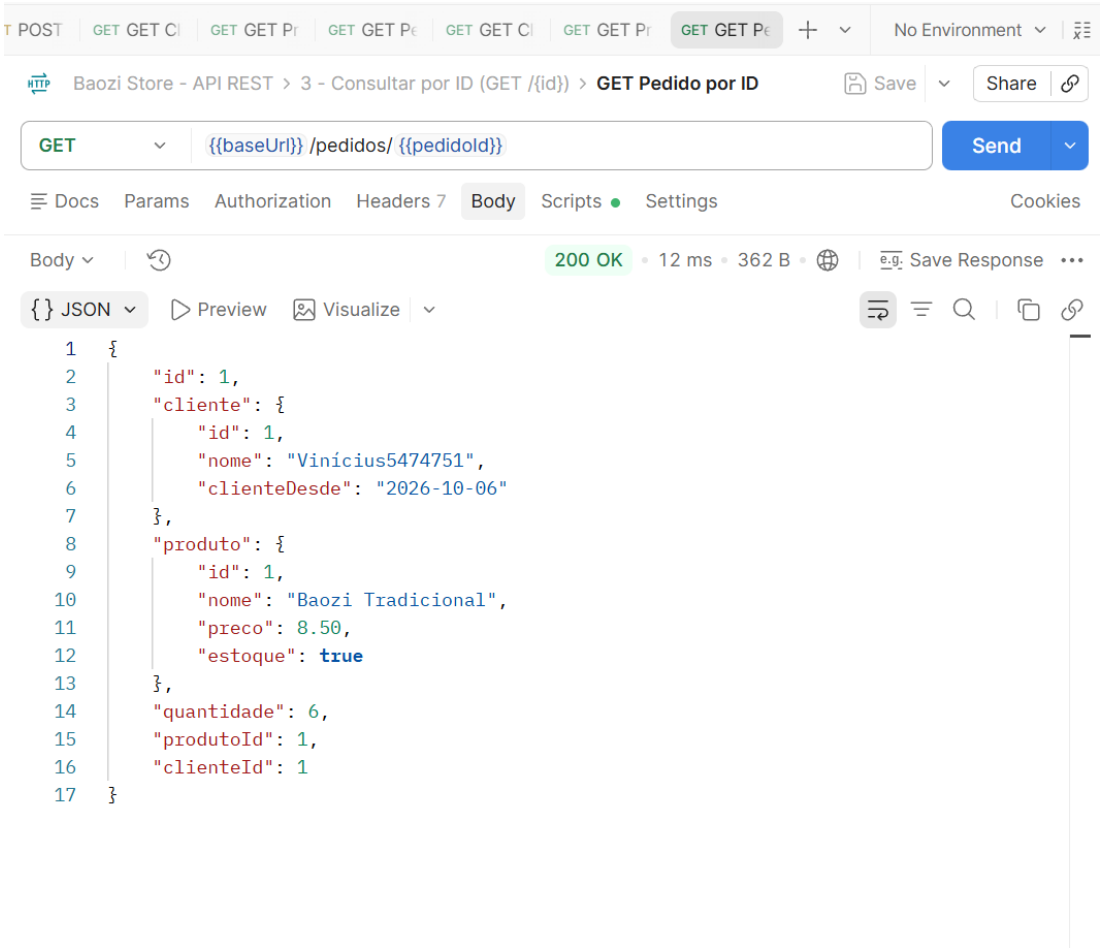
4.3.2 GET – Consultar Produto por ID

Requisição: `GET /produtos/1`
Resultado esperado: status 200 OK; produto “Baozi Tradicional”.



4.3.3 GET – Consultar Pedido por ID

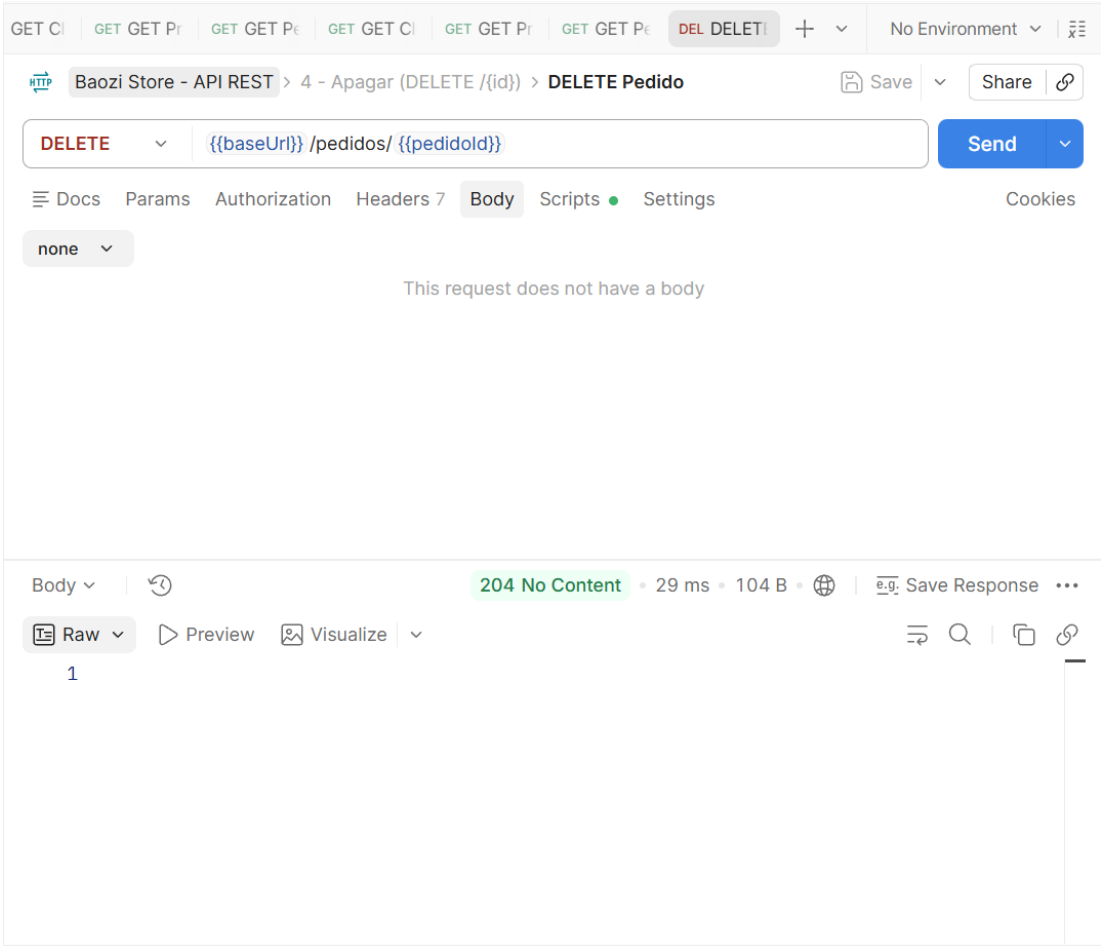
Requisição: `GET /pedidos/1`
Resultado esperado: status 200 OK; pedido com 6 unidades.



4.4 DELETE

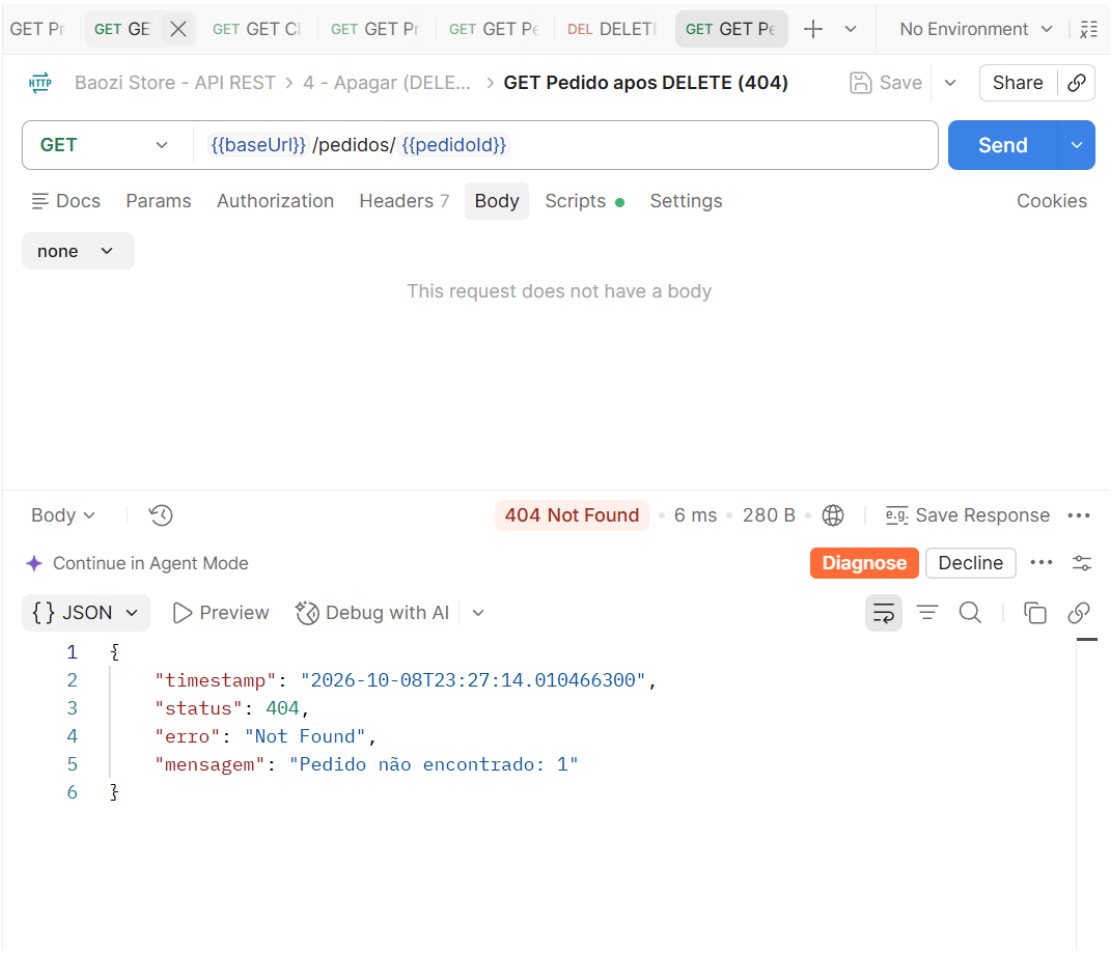
4.4.1 DELETE – Apagar Pedido

Requisição: `DELETE /pedidos/1`
Resultado esperado: status 204 No Content (sem corpo).



4.4.2 Confirmação – GET do pedido apagado

Requisição: `GET /pedidos/1`
Resultado esperado: status 404 Not Found com mensagem “Pedido não encontrado: 1”, comprovando a exclusão.



5. Link do repositório (código-fonte)

Repositório do projeto: https://github.com/Viniex/baozi-store

Como executar:

```
git clone https://github.com/Viniex/baozi-store
cd baozi-store
mvn spring-boot:run

# (opcional) usando MySQL:
mvn spring-boot:run -Dspring-boot.run.profiles=mysql
```

A API sobe em http://localhost:8080. Requisitos: JDK 17+ e Maven (ou use o Maven integrado da IDE).